

**Kristiania University College**  
**PGR211- Advance Programming for Data Science**  
**Worksheet 3**

1. Write a program using a loop that fills an empty list called **values** with ten random numbers between 1 and 100.
2. Write programs using **for loops** (without the use of the range function) that iterate over the list generated in Exercise 1 (**values**) and do the following tasks.
  - a. Printing all elements of the list in a single row, separated by spaces.
  - b. Computing the sum of all elements in the list.
  - c. Computing how many elements in the list are even numbers.
3. Consider the list generated in Exercise 1 (**values**) and separate the even and odd numbers into two different lists **even\_values**, **odd\_values**.
4. Consider the list generated in Exercise 1 (**values**) and remove the odd elements from the list.
5. Write a program that finds maximum, minimum, and average of the elements of a list.
6. Write a program that reads ten integers into a list, and sends the list to a function to display the elements in the opposite order from which they were entered.
7. Write a program that reads five integer numbers and adds them to a list. Then write functions that accept the list as an argument to perform the following tasks for the list.
  - a. Swap the first and last elements in the list.
  - b. Shift all elements by one to the right and move the last element into the first position. For example, **1 4 9 16 25** would be transformed into **25 1 4 9 16**.
  - c. Replace all even elements with 0.
  - d. Remove the middle element of the list.

8. Consider the following lists:

L1 = [1, 2, 3, 4]

L2 = [1, 2, 5, 6]

Write a function (**def remove\_dups (L1, L2)**) to take the lists as arguments and remove the elements from L1 if L2 has the same elements too.

9. Write a program that counts the elements in a tuple until an element is a list.
10. Write a function ***def equals (a, b)*** that checks whether two tuples have the same elements in the same order.
11. Write a function ***def read\_DOB ()*** that asks the user to enter his/her birthday (Day, Month, Year) and returns them as a tuple. Then the program prints the date of birth in ***DD/MM/YYYY*** format.
12. Write a program that removes empty tuple(s) from a list of tuples.
13. Write a program that removes an item from a tuple.
14. Given the set definitions below, answer the following questions:  
set1 = {1, 2, 3, 4, 5}  
set2 = {2, 4, 6, 8}  
set3 = {1, 5, 9, 13, 17}
- Is set1 a subset of set2?
  - Is the intersection of set1 and set3 empty?
  - What is the result of performing set union on set1 and set2?
  - What is the result of performing set intersection on set2 and set3?
  - What is the result of performing the set difference on set1 and set2 (set1 – set2)?
15. Given a dictionary  
grade\_count = {"A": 8, "D": 3, "B": 15, "F": 2, "C": 6}
- write the python statement(s) to print:
- All the keys.
  - All the values.
  - All the key and value pairs.
  - All the key and value pairs in key order.
  - The average value.
  - A chart similar to the following in which each row contains a key followed by a number of asterisks equal to the key's data value. The rows should be printed in key order, as shown below.
- A: \*\*\*\*\*  
B: \*\*\*\*\*  
C: \*\*\*\*\*  
D: \*\*\*  
F: \*\*

16. Write a program that keeps a dictionary in which both keys and values are strings—names of students and their course grades. Prompt the user of the program to add or remove students, to modify grades, or to print all grades. The printout should be sorted by name formatted like this:

Carl: B+

Francine: A

Joe: C

Sarah: A

17. Consider a dictionary of different usernames (keys) and passwords (values). Write a function called **accept\_login ()** with three parameters: the dictionary, a username, and a password. The function should return **True** if the user exists and the password is correct, then the message “**Login Successful**” is printed. Otherwise, the function return **False**, then the message “**Login Failed**” is printed.

18. Write a function **def one\_to\_one (d)** that takes a dictionary **d** and return True if every value in **d** has only one corresponding key. For example if **d = {'a': 4, 'b': 5, 'c': 3}**, the function returns True but if **d = {'a': 2, 'b': 4, 'c': 2}**, the function returns False.